🔧 Device Service

> 🚀 **Built for:** Remote device management, configuration, and over-the-air updates
> 🔧 **Tech Stack:** GraphQL, AWS Lambda, AWS IAM, Amazon Cognito
> 📊 **Data Sources:** AWS IoT Core, Amazon S3, Amazon SNS

🏷️ Enums

🔧 Commands

Device control commands.

Not all commands are available for all device types. Some commands also require specific device features, hardware, or additional parameters.

Value	Description	Det
`ADJUST_DURATION`	⏱️ Adjusts the active session duration	Rec `mir`
`REMAINING_TIME`	⏰ Updates remaining session time	Rec `mir`
`AFTER_HEATER`	🔥 Controls the after-heater function	Rec on/ stat
`EXT_SWITCH`	🔌 Controls an external	Rec `inc`

On this page

- 🏷️ Enums
  - 🔧 Commands
  - 🔄 OtaState
  - ☁️ CloudLoggingState
  - 🏢 OrganizationUpdate
  - 👁️ VisibleReason
- 🔵 GraphQL
  - 🔍 Queries
    - 🔍 `devicesGet`
    - 🔍 `devicesSearch`
    - 📊 `devicesStatesGet`
    - 🏷️ `devicesTagsList`
    - 🔐 `devicesTokenExists`
    - 🔒 `devicesEncrypt`
    - 🏢 `organizationsDevicesList`
    - 📋 `organizationsContractsDevicesList`
    - 👤 `usersDevicesList`
    - 🔧 `operatorsDevicesList`
    - 📱 `devicesMetadataGet`
    - 🔄 `otaUpdatesList`
    - 📊 `otaUpdatesStatesList`
    - 🏢 `devicesFleetStatusGet`
    - 📦 `otaUpdatesBatchList`
  - 🖍️ Mutations
    - 📤 `devicesCommandsSend`
    - 🗑️ `devicesDelete`
    - 📊 `devicesStatesUpdate`
    - 🖍️ `devicesUpdate`
    - 🏷️ `devicesTagsUpdate`
    - 🔄 `devicesOtaUpdatesStart`
    - ❌ `devicesOtaUpdatesCancel`

EXT_SWITCH	switch output	on/ stat
FAN	💨 Controls fan state	Rec on/ stat
HEATER	🔥 Controls heater state	Rec on/ stat
IR_HEATER	🌡️ Controls infrared heater mode	Rec moc
LIGHTS	💡 Controls lighting state	Rec on/ stat
RESTART	🔄 Requests device restart	No adc par
SAUNA	🧖 Controls sauna state	Rec on/ stat
STEAMER	💨 Controls steamer state	Rec on/ stat
TRACE_LOG	📝 Controls device trace logging	Rec on/ stat ma inte for diag
UPDATE	🔄 Requests a device software update	No adc par

`VAPORIZER`	💧 Controls vaporizer state	Rec on/ stat

🔄 OtaState

OTA update execution states:

Value	Description	Status
`IDLE`	📋 No update in progress	Ready
`IN_PROGRESS`	⚡ Update running	Active
`DONE`	✅ Update completed	Success
`CANCELLED`	❌ Update cancelled	Failed

☁️ CloudLoggingState

Remote logging configuration states:

Value	Description	Usage
`DISABLED`	🚫 No remote logging	Default
`MANUAL`	📝 Manual log collection	On-demand
`CONTINUOUS`	📊 Continuous logging	Real-tim monitori

🏢 OrganizationUpdate

Device organization membership changes:

Value	Description	Action
`ADD`	➕ Add device to organization	Join
`REMOVE`	➖ Remove device from organization	Leave

👁️ VisibleReason

Device access visibility levels:

Value	Description	Acces Level
`Organization`	🏢 Organization member	Org acces
`Contract`	📋 Contract access	Contr acces
`CanSee`	👀 Direct visibility	Direct acces
`Unknown`	❓ Unknown reason	Limite acces

🔵 GraphQL

The Device Service GraphQL API is the full API surface for device management and control. The REST API documented below is a subset of these capabilities for common operations.

Note: All GraphQL requests require a Cognito ID token in the `Authorization: Bearer <idToken>` header. See the API

Bearer <idToken> header. See the API Overview section for authentication setup.

Base URL: Get the GraphQL endpoint from the endpoints configuration: `endpoints.GraphQL.device.https`

🔍 Queries

🔍 `devicesGet`

> Retrieves a specific device by ID.

Arguments:

Parameter	Type	Required	De
`deviceId`	`ID!`	✅	The the

Returns: `Device` - Device information with attributes and roles

Example:

```
query GetDevice {
  devicesGet(deviceId: "DEVICE-ABC123
    id
    type
    attr {
      key
      value
    }
    roles
    via
  }
}
```

🔍 `devicesSearch`

> Searches for devices using a query string.

Arguments:

Parameter	Type	Required

Parameter	Type	Required	
query	String!	✅	
nextToken	String		
maxResults	Int		

Returns: `Devices` - List of matching devices with pagination

Example:

```
query SearchDevices {
  devicesSearch(query: "type:sauna",
    devices {
      id
      type
      attr {
        key
        value
      }
    }
    nextToken
  }
}
```

---

devicesStatesGet

> Gets the current state of a device shadow.

Arguments:

Parameter	Type	Required	
deviceId	ID!	✅	
shadowName	String		

Returns: `DeviceState` - Device shadow state with desired/reported values

Example:

```
query GetDeviceState {
  devicesStatesGet(deviceId: "DEVICE-
    deviceId
    shadowName
    desired
    reported
    timestamp
    version
    connectionState {
      connected
      updatedTimestamp
    }
  }
}
```

🏷️ devicesTagsList

> Lists all tags for a device.

Arguments:

Parameter	Type	Required	De
deviceId	ID!	✅	The the

Returns: `[String!]!` - Array of tag strings

Example:

```
query GetDeviceTags {
  devicesTagsList(deviceId: "DEVICE-A
}
```

🔐 devicesTokenExists

> Checks if a device token exists in secrets manager.

Arguments:

Parameter	Type	Required	De
deviceId	ID!	✅	The the

Returns: `Boolean` - True if token exists

Example:

```
query CheckDeviceToken {
  devicesTokenExists(deviceId: "DEVIC
}
```

🔒 devicesEncrypt

> Encrypts a message for a specific device.

Arguments:

Parameter	Type	Require
deviceId	ID	
certificateArn	String	
message	String!	✅

Returns: `String` - Encrypted message

Example:

```
query EncryptMessage {
  devicesEncrypt(
    deviceId: "DEVICE-ABC123-XYZ789"
    message: "Hello, device!"
  )
}
```

🏢 organizationsDevicesList

> Lists all devices in an organization.

Arguments:

Parameter	Type	Require
`organizationId`	`ID!`	✅
`nextToken`	`String`	⚪
`maxResults`	`Int`	⚪
`recursive`	`Boolean`	⚪

Returns: `Devices` - List of organization devices with pagination

Example:

```
query ListOrgDevices {
  organizationsDevicesList(
    organizationId: "ORG-PROD-001"
    maxResults: 50
  ) {
    devices {
      id
      type
      attr {
        key
        value
      }
    }
    nextToken
  }
}
```

📋 `organizationsContractsDevicesList`

> Lists devices accessible through contracts.

Arguments:

Parameter	Type	Required
organizationId	ID!	✅

Returns: `Devices` - List of contract-accessible devices

Example:

```
query ListContractDevices {
  organizationsContractsDevicesList(c
    devices {
      id
      type
      attr {
        key
        value
      }
    }
    nextToken
  }
}
```

---

usersDevicesList

> Lists devices owned by the calling user.

Arguments:

Parameter	Type	Required	D
nextToken	String		P t

Returns: `Devices` - List of user's devices with pagination

Example:

```
query ListMyDevices {
  usersDevicesList {
    devices {
      id
      type
```

```
      attr {
        key
        value
      }
    }
    nextToken
  }
}
```

🔧 `operatorsDevicesList`

> Lists devices with direct contract access.

Arguments:

Parameter	Type	Required	D
`nextToken`	`String`		P t

Returns: `Devices` - List of operator-accessible devices

Example:

```
query ListOperatorDevices {
  operatorsDevicesList {
    devices {
      id
      type
      attr {
        key
        value
      }
    }
    nextToken
  }
}
```

🧮 `devicesMetadataGet`

> Gets device metadata including owner and roles.

Arguments:

Parameter	Type	Required	De

deviceId	ID!	✅	The the

Returns: `DeviceMetadata` - Device metadata with owner and contact info

Example:

```
query GetDeviceMetadata {
  devicesMetadataGet(deviceId: "DEVIC
    deviceId
    owner
    roles
    contactName
    phoneCountryCode
    phoneNumber
  }
}
```

---

otaUpdatesList

> Lists available OTA updates.

Arguments:

Parameter	Type	Required	
nextToken	String		
deviceType	String		
hwVersion	String		

Returns: `OtaUpdates` - List of available OTA updates

Example:

```
query ListOtaUpdates {
  otaUpdatesList(deviceType: "sauna")
    otaUpdates {
      otaId
```

```
      firmwareVersion
      size
      description
      enabled
      deviceType
      hwVersion
    }
    nextToken
  }
}
```

---

otaUpdatesStatesList

> Lists OTA update states for devices.

Arguments:

Parameter	Type	Required
onlyActive	Boolean	
nextToken	String	

Returns: `OtaUpdateStates` - List of OTA update states

Example:

```
query ListOtaUpdateStates {
  otaUpdatesStatesList(onlyActive: tr
    otaUpdateStates {
      deviceId
      updateState
      progressPercent
      timestamp
    }
    nextToken
  }
}
```

---

devicesFleetStatusGet

> Gets fleet status for an organization.

Arguments:

Parameter	Type	Required
organizationId	ID!	✅

Returns: `DeviceFleetStatus` - Fleet status with device counts

Example:

```
query GetFleetStatus {
  devicesFleetStatusGet(organizationI
    fleetStatus {
      key
      value
    }
  }
}
```

📦 otaUpdatesBatchList

> Lists OTA update batch executions for an organization.

Arguments:

Parameter	Type	Required
organizationId	ID!	✅
nextToken	String	⚪

Returns: `OtaUpdateBatchExecutions` - List of batch executions with pagination

Example:

```
query ListOtaBatches {
  otaUpdatesBatchList(organizationId:
    otaBatchExecutions {
```

```
      id
      startDate
      currentCount
      maxCount
      executionStatus
      idle
      updating
      done
      failed
    }
    nextToken
  }
}
```

🖍️ Mutations

📤 devicesCommandsSend

> Sends a command to a device.

Arguments:

Parameter	Type	Required
deviceId	ID!	✅
command	Command!	✅
subId	String	🟣
params	AWSJSON	🟣

Returns: `CommandResponse` - Command execution result

Example:

```
mutation SendCommand {
  devicesCommandsSend(
    deviceId: "DEVICE-ABC123-XYZ789"
    command: { type: SAUNA }
    params: "{\"on\":1}"
```

```
    params: {{ on{ .1}
  ) {
    response
    failureReason
  }
}
```

---

devicesDelete

> Deletes a device.

Arguments:

Parameter	Type	Required	De
deviceId	ID!	✅	The the

Returns: `String` - Deletion confirmation

Example:

```
mutation DeleteDevice {
  devicesDelete(deviceId: "DEVICE-ABC
}
```

---

devicesStatesUpdate

> Updates device shadow state.

Arguments:

Parameter	Type	Required
deviceId	ID!	✅
state	AWSJSON!	✅
shadowName	String	

clientToken	String	

Returns: `AWSJSON` - Updated state

Example:

```
mutation UpdateDeviceState {
  devicesStatesUpdate(
    deviceId: "DEVICE-ABC123-XYZ789"
    state: "{\"desired\": {\"temp\":
    shadowName: "C1"
  )
}
```

---

devicesUpdate

> Updates device attributes.

Arguments:

Parameter	Type	R
deviceId	ID!	
attributes	[AttributeInput!]!	

Returns: `Device` - Updated device

Example:

```
mutation UpdateDevice {
  devicesUpdate(
    deviceId: "DEVICE-ABC123-XYZ789"
    attributes: [
      { key: "name", value: "Main Sau
      { key: "location", value: "Buil
    ]
  ) {
    id
    type
    attr {
      key
      value
    }
  }
```

```
}
```

🏷️ devicesTagsUpdate

> Updates device tags.

Arguments:

Parameter	Type	Required
deviceId	ID!	✅
tags	[String!]!	✅

Returns: `[String!]!` - Updated tags

Example:

```
mutation UpdateDeviceTags {
  devicesTagsUpdate(
    deviceId: "DEVICE-ABC123-XYZ789"
    tags: ["production", "sauna", "bu
  )
}
```

🔄 devicesOtaUpdatesStart

> Starts OTA update for a device.

Arguments:

Parameter	Type	Required	De
deviceId	ID!	✅	The the
otaId	ID!	✅	The upd

Returns: `Boolean` - Success status

Example:

```
mutation StartOtaUpdate {
  devicesOtaUpdatesStart(
    deviceId: "DEVICE-ABC123-XYZ789"
    otaId: "ota-456"
  )
}
```

❌ devicesOtaUpdatesCancel

> Cancels OTA update for a device.

Arguments:

Parameter	Type	Required	De
deviceId	ID!	✅	The the

Returns: `Boolean` - Success status

Example:

```
mutation CancelOtaUpdate {
  devicesOtaUpdatesCancel(deviceId: "
}
```

🏢 organizationsDevicesMove

> Moves a device between organizations.

Arguments:

Parameter	Type	Required
deviceId	ID!	✅
organizationId	ID	🟣
subId	String	🟣

Returns: `Device` - Updated device (null if unmanaged)

Example:

```
mutation MoveDevice {
  organizationsDevicesMove(
    deviceId: "DEVICE-ABC123-XYZ789"
    organizationId: "ORG-NEW-001"
  ) {
    id
    type
    attr {
      key
      value
    }
  }
}
```

---

devicesRemoteLoggingSet

> Sets remote logging state for a device.

Arguments:

Parameter	Type
`deviceId`	`ID!`
`cloudLogging`	`CloudLoggingState!`

Returns: `Boolean` - Success status

Example:

```
mutation SetRemoteLogging {
  devicesRemoteLoggingSet(
    deviceId: "DEVICE-ABC123-XYZ789"
    cloudLogging: CONTINUOUS
  )
}
```

---

devicesTokenSet

> Saves device token to secrets manager.

Arguments:

Parameter	Type	Required
deviceId	ID!	✅
token	String!	✅
mac	String	⚪

Returns: `Boolean` - Success status

Example:

```
mutation SetDeviceToken {
  devicesTokenSet(
    deviceId: "DEVICE-ABC123-XYZ789"
    token: "device-token-string"
    mac: "AA:BB:CC:DD:EE:FF"
  )
}
```

🔗 devicesPair

> Pairs a device with Home2Net cloud.

Arguments:

Parameter	Type	Required
mac	String!	✅

Returns: `String` - Pairing result

Example:

```
mutation PairDevice {
```

```
  devicesPair(mac: "AA:BB:CC:DD:EE:FF
}
```

🚀 otaUpdatesBatchStart

> Starts OTA update batch for devices in an organization.

Arguments:

Parameter	Type	Require
organizationId	ID!	✅
query	String!	✅
otaId	ID!	✅
maxCount	Int!	✅
dailyMaxCount	Int	⚪

Returns: `ID` - Batch execution ID

Example:

```
mutation StartOtaBatch {
  otaUpdatesBatchStart(
    organizationId: "ORG-PROD-001"
    query: "type:sauna"
    otaId: "ota-456"
    maxCount: 100
    dailyMaxCount: 10
  )
}
```

```
}
```

---

otaUpdatesBatchStop

> Stops OTA update batch execution.

Arguments:

Parameter	Type	Required
organizationId	ID!	✅
id	ID!	✅

Returns: `Boolean` - Success status

Example:

```
mutation StopOtaBatch {
  otaUpdatesBatchStop(
    organizationId: "ORG-PROD-001"
    id: "batch-execution-123"
  )
}
```

---

organizationsOtaUpdatesStart

> Starts OTA updates for all devices in an organization.

Arguments:

Parameter	Type	Required
organizationId	ID!	✅
otaId	ID!	✅

Returns: `Boolean` - Success status

Example:

```
mutation StartOrgOtaUpdates {
  organizationsOtaUpdatesStart(
    organizationId: "ORG-PROD-001"
    otaId: "ota-456"
  )
}
```

❌ `organizationsOtaUpdatesCancel`

> Cancels OTA updates for all devices in an organization.

Arguments:

Parameter	Type	Required
`organizationId`	`ID!`	✅

Returns: `Boolean` - Success status

Example:

```
mutation CancelOrgOtaUpdates {
  organizationsOtaUpdatesCancel(organ
}
```

📋 `organizationsContractsAddDevice`

> Creates a new contract for a device between organizations.

Arguments:

Parameter	Type	Req
`organizationId`	`ID!`	✅

deviceSerialNumber	String!	✅
userEmail	String!	✅

Returns: `DeviceContractResult` - Contract creation result

Example:

```
mutation CreateDeviceContract {
  organizationsContractsAddDevice(
    organizationId: "ORG-PROD-001"
    deviceSerialNumber: "SN123456789"
    userEmail: "user@example.com"
  ) {
    contractId
    contractName
    deviceId
  }
}
```

📍 devicesLocationStore

> Stores device GPS coordinates and location information as device attributes.

Arguments:

Parameter	Type	Required	D
deviceId	ID!	✅	T t
latitude	Float!	✅	(l
longitude	Float!	✅	(l
accuracy	Float!	✅	L a r

Returns: `Boolean` - Success status

Example:

```
mutation StoreDeviceLocation {
  devicesLocationStore(
    deviceId: "DEVICE-ABC123-XYZ789"
    latitude: 60.1699
    longitude: 24.9384
    accuracy: 10.5
  )
}
```

💻 HTTP Request Examples

The following examples show how to make GraphQL queries and mutations using HTTP. Each example includes the complete authentication flow. For detailed authentication setup, token refresh, and error handling, see the API Overview section.

🟧 Using JavaScript/fetch

```
// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiGenerics = endpoints.Res
const graphqlEndpoint = endpoints.Gra

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

// Example query
const query = `
  query GetDevice {
    devicesGet(deviceId: "DEVICE-ABC1
      id
      type
      attr {
        key
        value
      }
      roles
```

```
      via
    }
  }
`;

const graphqlResponse = await fetch(g
  method: 'POST',
  headers: {
    'Content-Type': 'application/json
    'Authorization': `Bearer ${idToke
  },
  body: JSON.stringify({ query })
});

const data = await graphqlResponse.js
console.log(data);
```

Using Python/requests

```
import requests

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_generics = endpoints["RestAp
graphql_endpoint = endpoints["GraphQL

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]

# Example query
query = """
query GetDevice {
  devicesGet(deviceId: "DEVICE-ABC123
    id
    type
    attr {
      key
      value
    }
    roles
    via
  }
}
"""

response = requests.post(
    graphql_endpoint,
    headers={
        'Content-Type': 'application/
        'Authorization': f"Bearer {id
    },
```

```
    json={ query : query}
)

data = response.json()
print(data)
```

🔧 Using cURL

```
# Get endpoints and authenticate
ENDPOINTS=$(curl -sS "https://api.har
REST_API_GENERICS=$(echo "$ENDPOINTS"
GRAPHQL=$(echo "$ENDPOINTS" | jq -r '

TOKEN_RESPONSE=$(curl -sS -H "Content
  -X POST "$REST_API_GENERICS/auth/to
  --data '{"username":"your-username"

ID_TOKEN=$(echo "$TOKEN_RESPONSE" | j

QUERY='{"query":"query GetDevice {\\n

curl -sS -H "Content-Type: applicatio
  -H "Authorization: Bearer $ID_TOKEN
  -X POST "$GRAPHQL" \
  --data "$QUERY" | jq '.'
```

📡 Subscriptions

> ⚠️ **Important**: Subscriptions are more complex than queries/ mutations as they require WebSocket connections.

🔧 Setup Requirements

JavaScript/Node.js:

```
npm install aws-appsync aws-appsync-a
```

Note: `aws-appsync` requires `graphql` version 14.x or 15.0.0–15.3.0 (not 16+). If you encounter compatibility issues, install with: `npm install aws-appsync aws-appsync-auth-link graphql@14 graphql-tag`

Python:

```
pip install requests websocket-client
```

Note: Use the Cognito IdToken obtained via the REST API (see API Overview). Get endpoints from the Endpoints API; the client URL comes from `endpoints.GraphQL.device.https`. The `receiver` must be the JWT claim `cognito:username` from your IdToken.

📊 devicesStatesUpdateFeed

> Real-time feed of device state updates.

Arguments:

Parameter	Type	Required	De
`receiver`	`ID!`	✅	The the

Returns: `DeviceStateUpdateNotice` - Real-time device state updates

Example:

```
subscription DeviceStateUpdates {
  devicesStatesUpdateFeed(receiver: "
    receiver
    item {
      deviceId
      desired
      reported
      timestamp
      connectionState {
        connected
        updatedTimestamp
      }
    }
  }
}
```

🔄 otaUpdatesStatesUpdateFeed

> Real-time feed of OTA update state changes

changes.

Arguments:

Parameter	Type	Required	De
`receiver`	`ID!`	✅	The the

Returns: `OtaUpdateStateNotice` - Real-time OTA update notifications

Example:

```
subscription OtaUpdateStates {
  otaUpdatesStatesUpdateFeed(receiver
    receiver
    item {
      deviceId
      updateState
      progressPercent
      timestamp
    }
  }
}
```

🏢 devicesOrganizationsUpdateFeed

> Real-time feed of device organization changes.

Arguments:

Parameter	Type	Required	De
`receiver`	`ID!`	✅	The the

Returns: `DeviceOrganizationUpdateNotice` - Real-time organization updates

Example:

```
subscription DeviceOrganizationUpdate
  devicesOrganizationsUpdateFeed(rece
    receiver
```

```
    deviceId
    organizationId
    updateType
    timestamp
  }
}
```

✏️ devicesAttributesUpdateFeed

> Real-time feed of device attribute changes.

Arguments:

Parameter	Type	Required	De
receiver	ID!	✅	The the

Returns: DeviceAttributesUpdateNotice - Real-time attribute updates

Example:

```
subscription DeviceAttributesUpdates
  devicesAttributesUpdateFeed(receive
    receiver
    deviceId
    attributes {
      key
      value
    }
    timestamp
  }
}
```

📦 otaUpdatesBatchFeed

> Real-time feed of OTA batch update notifications.

Arguments:

Parameter	Type	Required	De
receiver	ID!	✅	The the

Returns: `OtaUpdatesBatchNotice` - Real-time batch update notifications

Example:

```
subscription OtaBatchUpdates {
  otaUpdatesBatchFeed(receiver: "user
    item {
      organizationId
      id
      currentCount
      executionStatus
      idle
      updating
      done
      failed
    }
  }
}
```

📊 Subscription Examples

The following examples show complete subscription setup including authentication. For detailed authentication setup and token management, see the API Overview section.

🟧 Using JavaScript/Node.js

```
import { AWSAppSyncClient } from "aws
import { AUTH_TYPE } from "aws-appsyn
import gql from "graphql-tag";

// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiGenerics = endpoints.Res
const graphqlEndpoint = endpoints.Gra

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

// Extract user ID from JWT token (re
const tokenPayload = JSON.parse(Buffe
```

```
const userId = tokenPayload['cognito:

// Create AppSync client
const client = new AWSAppSyncClient({
  url: graphqlEndpoint,
  region: "eu-central-1",
  auth: {
    type: AUTH_TYPE.AWS_LAMBDA,
    token: async () => `Bearer ${idTo
  },
  disableOffline: true,
});

// Subscribe to device state updates
const subscription = client.subscribe
  query: gql`
    subscription DeviceStateUpdates {
      devicesStatesUpdateFeed(receive
        receiver
        item {
          deviceId
          desired
          reported
          timestamp
          connectionState {
            connected
            updatedTimestamp
          }
        }
      }
    }
  `
});

subscription.subscribe({
  next: (data) => console.log("Receiv
  error: (error) => console.error("Er
});
```

Using Python/requests

```
import json
import base64
import websocket
import requests
from uuid import uuid4

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_generics = endpoints["RestAp
graphql_endpoint = endpoints["GraphQL

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()
```

```
).json()

id_token = tokens["idToken"]

def header_encode(header_obj):
    """Encode header using Base 64"""
    return base64.b64encode(json.dump

# Extract user ID from JWT token
token_payload = json.loads(base64.b64
user_id = token_payload['cognito:user

# Build WebSocket URL and host
wss_url = graphql_endpoint.replace('h
host = graphql_endpoint.replace('http

# Generate subscription ID
sub_id = str(uuid4())

# Create JWT authentication header
auth_header = {
    'host': host,
    'Authorization': f"Bearer {id_tok
}

# GraphQL subscription
gql_subscription = json.dumps({
    'query': f'subscription DeviceSta
    'variables': {}
})

# WebSocket event callbacks
def on_message(ws, message):
    message_object = json.loads(messa
    message_type = message_object['ty

    if message_type == 'connection_ac
        # Register subscription
        register = {
            'id': sub_id,
            'payload': {
                'data': gql_subscript
                'extensions': {'autho
            },
            'type': 'start'
        }
        ws.send(json.dumps(register))

    elif message_type == 'start_ack':
        print("✅ Subscription regis

    elif message_type == 'data':
        print("✅ Received subscripti
        # Stop subscription
        ws.send(json.dumps({'type': '

def on_open(ws):
    ws.send(json.dumps({'type': 'conn

# Create WebSocket connection
```

```
connection_url = wss_url + '?header='
ws = websocket.WebSocketApp(
    connection_url,
    subprotocols=['graphql-ws'],
    on_open=on_open,
    on_message=on_message
)

# Run WebSocket (use proper threading
ws.run_forever()
```

📋 Types

🔧 Device

> Device representation

Field	Type	Required
id	String!	✅
type	String!	✅
attr	[Attribute!]!	✅
roles	[String!]!	✅
via	VisibleReason!	✅

📋 Devices

> Paginated device list

Field	Type	Required
devices	[Device!]!	✅
nextToken	String	🟣

📊 DeviceState

Device shadow state

Field	Type
deviceId	ID!
shadowName	String
desired	AWSJSON
reported	AWSJSON
timestamp	Float
version	Int
clientToken	String
connectionState	DeviceConnectionSt
metadata	AWSJSON

DeviceMetadata

Device metadata

Field	Type	Requ
deviceId	ID!	✅
owner	String	
roles	[String]	

contactName	String	
phoneCountryCode	String	
phoneNumber	String	

CommandResponse

Command execution result

Field	Type	Require
response	Boolean!	✅
failureReason	String	

OtaUpdate

OTA update information

Field	Type	Re
otaId	String!	✅
firmwareVersion	String!	✅
size	Int	
description	String	
filename	String	

enabled	Boolean	
urlExpirationSeconds	Int	
deviceType	String	
hwVersion	String	
betaTesting	Boolean	

📦 OtaUpdates

> Paginated OTA updates list

Field	Type	Requi
otaUpdates	[OtaUpdate!]!	✅
nextToken	String	

📊 OtaUpdateState

> OTA update state

Field	Type
batchKey	String
deviceId	ID!
updateFirmwareVersion	String

`updateState`	`OtaState`	
`resultCode`	`Int`	
`progressPercent`	`Int`	
`timestamp`	`String`	

🏢 DeviceFleetStatus

> Fleet status

Field	Type
`fleetStatus`	`[DeviceFleetStatusAttr`

🔗 Attribute

> Device attribute

Field	Type	Required	Desc
`key`	`String!`	✅	Attri key
`value`	`String`	🟣	Attri valu

🔌 DeviceConnectionState

> Device connection state

Field	Type	Requ
`connected`	`Boolean!`	✅
`updatedTimestamp`	`String!`	✅

📦 OtaUpdateStates

> Paginated OTA update states list

Field	Type
otaUpdateStates	[OtaUpdateState!]!
nextToken	String

🚀 OtaBatchExecution

> OTA batch execution

Field	Type	Require
organizationId	ID!	✅
id	ID!	✅
startDate	String	⚪
stopDate	String	⚪
currentCount	Int	⚪
maxCount	Int	⚪
dailyMaxCount	Int	⚪
executionStatus	String	⚪
searchQuery	String	⚪

userId	String	
idle	Int	
updating	Int	
done	Int	
failed	Int	

📋 OtaUpdateBatchExecutions

> Paginated OTA batch executions list

Field	Type
otaBatchExecutions	[OtaBatchExecut
nextToken	String

🔑 DeviceFleetStatusAttribute

> Fleet status attribute

Field	Type	Required	Desc
key	String!	✅	Attri key
value	Int!	✅	Attri valu

📄 DeviceContractResult

> Device contract creation result

Field	Type	Required

contractId	ID!	✅
contractName	String!	✅
deviceId	ID!	✅

📋 Sample Responses

✅ Successful Query Response

```
{
  "data": {
    "devicesGet": {
      "id": "DEVICE-ABC123-XYZ789",
      "type": "sauna",
      "attr": [
        { "key": "name", "value": "Ma
        { "key": "location", "value":
      ],
      "roles": ["owner"],
      "via": "Organization"
    }
  }
}
```

❌ Error Response

```
{
  "errors": [
    {
      "message": "Device not found",
      "locations": [{"line": 3, "colu
      "path": ["devicesGet"],
      "extensions": {
        "code": "DEVICE_NOT_FOUND",
        "exception": {
          "stacktrace": ["Error: Devi
        }
      }
    }
  ],
  "data": null
}
```

📄 Pagination

For paginated results, use the `nextToken` from the response in subsequent requests:

```
query GetMoreDevices {
  devicesSearch(
    query: "type:sauna"
    maxResults: 50
    nextToken: "eyJsYXN0RXZhbHVhdGVkS
  ) {
    devices {
      id
      type
      attr {
        key
        value
      }
    }
    nextToken
  }
}
```

🌎 Configuration Reference: Harvia Endpoints API

📝 Note: Always fetch the latest configuration to ensure you're using the current endpoints, regions, and client IDs. The configuration may change over time.

🌐 REST API

The REST API provides a subset of the full Device Service GraphQL API for common integration flows.

Note: All REST API endpoints require a Cognito ID token in the `Authorization: Bearer <idToken>` header. See the API Overview section for authentication setup.

Base URL: Get the REST API base URL from the endpoints configuration:

```
endpoints.RestApi.device.https
```

endpoints.RestApi.device.https

📋 GET /devices

> Retrieves a paginated list of devices owned by the authenticated user.

Query Parameters:

Parameter	Type	Required	
maxResults	number		
nextToken	string		

Success Response (200):

```
{
  "devices": [
    {
      "deviceId": "DEVICE-ABC123-XYZ7
      "type": "sauna",
      "attr": [
        { "key": "name", "value": "Ma
        { "key": "location", "value":
      ],
      "roles": ["owner"],
      "via": "Organization"
    }
  ],
  "nextToken": "eyJwYWdlIjoiMiJ9"
}
```

Error Response:

```
{
  "error": "string",
  "message": "string"
}
```

Notes:

- Pagination: When `nextToken` is returned, pass it back to fetch the next page.
- Authorization is based on the caller's ID token and service authorization rules.

📤 `POST /devices/command`

> Sends a command to a device and waits for acknowledgement.

Request Body:

Parameter	Type	Require
`deviceId`	`string`	✅
`cabin`	`object`	🟣
`cabin.id`	`string`	🟣
`cabin.name`	`string`	🟣

command	object	✅
command.type	enum	✅
command.state	string| boolean| number	
command.params	object	

Success Response (200):

```
{
  "handled": true
}
```

Error Response:

```
{
  "error": "string",
  "message": "string",
  "handled": false,
  "failureReason": "Device unavailabl
}
```

Notes:

- Persists the command, publishes it to the device, and waits for an

to the device, and waits for an acknowledgement.

- If the device doesn't respond in time, returns `504 Gateway Timeout` with `{ handled: false, failureReason: "Device unavailable" }`.
- If neither `cabin.id` nor `cabin.name` is provided, `cabin.id` defaults to `C1`.

📊 `GET /devices/state`

> Retrieves the current state of a device shadow (named shadow, default depends on device type).

Query Parameters:

Parameter	Type	Required	D
`deviceId`	`string`	✅	D i (T N
`subId`	`string`	🟣	(s i (N e v
`cabinName`	`string`	🟣	F c r N e v

Success Response (200):

```
{
  "deviceId": "DEVICE-ABC123-XYZ789",
  "shadowName": "C1",
  "state": {
    "temp": 78,
    "targetHum": 38
  },
  "version": 123,
  "timestamp": 1735689600000,
  "metadata": {
    "state": {}
  },
  "connectionState": {
    "connected": true,
    "updatedTimestamp": 1735689600000
  }
}
```


Error Response:


```
{
  "error": "string",
  "message": "string"
}
```


Notes:

- Returns the selected named shadow state (not classic shadow, except for Sauna sensor). The `state` field contains reported data from the device.
- If neither `subId` nor `cabinName` is provided:
  - For most devices (e.g., Fenix), `subId` defaults to `C1`
  - For Sauna sensor devices, defaults to `classic` shadow

🌡️ `PATCH /devices/target`

> Updates the target temperature and/or humidity for a device cabin.

Request Body:

Parameter	Type	Required

deviceId	string	✅
temperature	number	
humidity	number	
cabin	object	
cabin.id	string	
cabin.name	string	

Success Response (200):

```
{
  "deviceId": "DEVICE-ABC123-XYZ789",
  "shadowName": "C1",
  "updated": {
    "temperature": 22,
    "humidity": 50
  }
```

```
}
```

Error Response:

```
{
  "error": "string",
  "message": "string"
}
```

Notes:

- Supply at least one of `temperature` or `humidity`; both can be provided to update together.
- If neither `cabin.id` nor `cabin.name` is provided, `cabin.id` defaults to `C1`.

`PATCH /devices/profile`

> Updates the active profile for a device cabin.

Request Body:

Parameter	Type	Required	
`deviceId`	`string`	✅	
`profile`	`string`	✅	
`cabin`	`object`		

`cabin.id`	`string`		
`cabin.name`	`string`		

Success Response (200):

```
{
  "deviceId": "DEVICE-ABC123-XYZ789",
  "shadowName": "C1",
  "activeProfile": 2,
  "profile": "eco"
}
```

Error Response:

```
{
  "error": "string",
  "message": "string"
}
```

Notes:

- If neither `cabin.id` nor `cabin.name` is provided, `cabin.id` defaults to `C1`.

💡 REST API Examples

Each example below shows the complete authentication flow. For detailed authentication setup, token refresh, and error handling, see the API Overview section

Overview section.

Using JavaScript/fetch

```
// Get endpoints and authenticate
const response = await fetch("https:/
const { endpoints } = await response.
const restApiBase = endpoints.RestApi
const restApiGenerics = endpoints.Res

const tokens = await fetch(`${restApi
  method: "POST",
  headers: { "Content-Type": "applica
  body: JSON.stringify({ username: "y
}).then(r => r.json());

const idToken = tokens.idToken;

async function call(method, path, bod
  const res = await fetch(`${restApiB
    method,
    headers: {
      Authorization: `Bearer ${idToke
      "Content-Type": "application/js
    },
    body: body === undefined ? undefi
  });
  if (!res.ok) throw new Error(`${res
  return res.json();
}

// List user's devices
const devices = await call("GET", `/d
console.log(devices);

// Send a device command
const commandResult = await call("POS
  deviceId: "DEVICE-ABC123-XYZ789",
  cabin: { id: "C1" },
  command: { type: "SAUNA", state: "c
});
console.log(commandResult);

// Get device state (cabin shadow via
const cabinStateById = await call(
  "GET",
  `/devices/state?deviceId=${encodeUR
);
console.log(cabinStateById);

// Update device target values
const updatedTarget = await call("PAT
  deviceId: "DEVICE-ABC123-XYZ789",
  cabin: { id: "C1" },
  temperature: 22,
  humidity: 50,
});
console.log(updatedTarget);
```

```
// Update active profile
const updatedProfile = await call("PA
  deviceId: "DEVICE-ABC123-XYZ789",
  cabin: { id: "C1" },
  profile: "eco",
});
console.log(updatedProfile);
```

Using Python/requests

```
import requests

# Get endpoints and authenticate
response = requests.get("https://api.
endpoints = response.json()["endpoint
rest_api_base = endpoints["RestApi"][
rest_api_generics = endpoints["RestAp

tokens = requests.post(
    f"{rest_api_generics}/auth/token"
    headers={"Content-Type": "applica
    json={"username": "your-username"
).json()

id_token = tokens["idToken"]

def call(method, path, body=None):
    res = requests.request(
        method,
        f"{rest_api_base}{path}",
        headers={"Authorization": f"B
        json=body if body else None
    )
    if not res.ok:
        raise Exception(f"{res.status
    return res.json()

# List user's devices
devices = call("GET", "/devices?maxRe
print(devices)

# Send a device command
command_result = call("POST", "/devic
    "deviceId": "DEVICE-ABC123-XYZ789
    "cabin": {"id": "C1"},
    "command": {"type": "SAUNA", "sta
})
print(command_result)

# Get device state
cabin_state = call("GET", "/devices/s
print(cabin_state)

# Update device target values
updated_target = call("PATCH", "/devi
    "deviceId": "DEVICE-ABC123-XYZ789
    "cabin": {"id": "C1"},
    "temperature": 22,
```

```
    "humidity": 50
})
print(updated_target)

# Update active profile
updated_profile = call("PATCH", "/dev
    "deviceId": "DEVICE-ABC123-XYZ789
    "cabin": {"id": "C1"},
    "profile": "eco"
})
print(updated_profile)
```

🔧 Using cURL

```
# Get endpoints and authenticate
ENDPOINTS=$(curl -sS "https://api.har
REST_API_BASE=$(echo "$ENDPOINTS" | j
REST_API_GENERICS=$(echo "$ENDPOINTS"

TOKEN_RESPONSE=$(curl -sS -H "Content
  -X POST "$REST_API_GENERICS/auth/to
  --data '{"username":"your-username"

ID_TOKEN=$(echo "$TOKEN_RESPONSE" | j

# List user's devices
curl -sS -H "Authorization: Bearer $I
     "$REST_API_BASE/devices?maxResul

# Send a device command
curl -sS -H "Authorization: Bearer $I
     -H "Content-Type: application/js
     -X POST \
     --data '{"deviceId":"DEVICE-ABC1
     "$REST_API_BASE/devices/command"

# Get device state
curl -sS -H "Authorization: Bearer $I
     "$REST_API_BASE/devices/state?de

# Update device target values
curl -sS -H "Authorization: Bearer $I
     -H "Content-Type: application/js
     -X PATCH \
     --data '{"deviceId":"DEVICE-ABC1
     "$REST_API_BASE/devices/target"

# Update active profile
curl -sS -H "Authorization: Bearer $I
     -H "Content-Type: application/js
     -X PATCH \
     --data '{"deviceId":"DEVICE-ABC1
     "$REST_API_BASE/devices/profile"
```
